Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2025**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

Solution for Project 3

Due date: Monday 07 April 2025, 18:00 CEST

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts). If you modified any provided build files, ensure they still work as expected.
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Task: Implementing the linear algebra functions and the stencil operators [50 Points]

1.1. Numerical method

We consider Fisher's reaction–diffusion equation on the square domain $\Omega = [0,1]^2$, where the population concentration $s(x,y,t)$ evolves under the combined effects of diffusion and logistic growth (eq. 1).

$$\frac{\partial s}{\partial t} = D\left(\frac{\partial^2 s}{\partial x^2} + \frac{\partial^2 s}{\partial y^2}\right)s + Rs(1-s) \tag{1}$$

The problem is complemented by constant Dirichlet boundary conditions $s = 0.1$ on $\partial\Omega$ and by an initial condition consisting of a localized perturbation, with $s = 0.2$ inside a small disk centered at $(1/4, 1/4)$ and $s = 0.1$ elsewhere.

The spatial domain is discretized on a uniform Cartesian grid with $(n+2)\times(n+2)$ points, where only the n^2 interior values are treated as unknowns. Spatial derivatives are approximated by a second-order central finite-difference scheme, yielding the standard five-point stencil, while time integration is performed using the implicit Euler method. At each time step, this results in a nonlinear algebraic system for the updated concentration field s^k.

The nonlinear system is solved with Newton's method applied to the residual equation $f(s^k) = 0$. At each Newton iteration, the corresponding linearized correction problem involving the Jacobian of the residual is solved approximately with a matrix-free Conjugate Gradient method, thereby avoiding explicit Jacobian inversion. The converged solution at time step k is then used as the initial guess for the subsequent time step.

We were asked to implement the linear algebra functions and the stencil operators, which are used in the Conjugate Gradient method and in the Newton's method, respectively.
One can look at the linear algebra functions in the file `linalg.cpp` and the stencil operators in the file `operators.cpp`.
In particular for the stencil operators we have to implement the residual function $f^k_{(i,j)}$ for the interior points, and for the points on the boundaries.
The residual function in the interior points is given by eq. 7 of the project handout. For the boundery points, we have to take into account the Dirichlet boundary conditions, obtaining:

$$\begin{aligned}
\text{East:} \quad & f^k_{(n+1,j)} = -(4+\alpha)s^k_{(n+1,j)} + s^k_{(n,j)} + 0.1 + s^k_{(n+1,j-1)} + s^k_{(n+1,j+1)} + \beta s^k_{(n+1,j)}(1 - s^k_{(n+1,j)}) + \alpha s^{k-1}_{(n+1,j)} \\
\text{West:} \quad & f^k_{(0,j)} = -(4+\alpha)s^k_{(0,j)} + 0.1 + s^k_{(1,j)} + s^k_{(0,j-1)} + s^k_{(0,j+1)} + \beta s^k_{(0,j)}(1 - s^k_{(0,j)}) + \alpha s^{k-1}_{(0,j)} \\
\text{North:} \quad & f^k_{(i,n+1)} = -(4+\alpha)s^k_{(i,n+1)} + s^k_{(i-1,n+1)} + s^k_{(i+1,n+1)} + 0.1 + s^k_{(i,n)} + \beta s^k_{(i,n+1)}(1 - s^k_{(i,n+1)}) + \alpha s^{k-1}_{(i,n+1)} \\
\text{South:} \quad & f^k_{(i,0)} = -(4+\alpha)s^k_{(i,0)} + s^k_{(i,1)} + 0.1 + s^k_{(i-1,0)} + s^k_{(i+1,0)} + \beta s^k_{(i,0)}(1 - s^k_{(i,0)}) + \alpha s^{k-1}_{(i,0)} \\
\text{NE corner:} \quad & f^k_{(n+1,n+1)} = -(4+\alpha)s^k_{(n+1,n+1)} + s^k_{(n,n+1)} + 0.1 + s^k_{(n+1,n)} + \beta s^k_{(n+1,n+1)}(1 - s^k_{(n+1,n+1)}) + \alpha s^{k-1}_{(n+1,n+1)} \\
\text{NW corner:} \quad & f^k_{(0,n+1)} = -(4+\alpha)s^k_{(0,n+1)} + 0.1 + s^k_{(1,n+1)} + s^k_{(0,n)} + \beta s^k_{(0,n+1)}(1 - s^k_{(0,n+1)}) + \alpha s^{k-1}_{(0,n+1)} \\
\text{SE corner:} \quad & f^k_{(n+1,0)} = -(4+\alpha)s^k_{(n+1,0)} + s^k_{(n,0)} + 0.1 + s^k_{(n+1,1)} + \beta s^k_{(n+1,0)}(1 - s^k_{(n+1,0)}) + \alpha s^{k-1}_{(n+1,0)} \\
\text{SW corner:} \quad & f^k_{(0,0)} = -(4+\alpha)s^k_{(0,0)} + 0.1 + s^k_{(1,0)} + s^k_{(0,1)} + \beta s^k_{(0,0)}(1 - s^k_{(0,0)}) + \alpha s^{k-1}_{(0,0)}
\end{aligned}$$

The `main.cpp` implements the time integration of the nonlinear system arising from the implicit discretization of Fisher's equation.
For each time step k, Newton's method is applied to solve the nonlinear system $f(s^k) = 0$. At iteration m, the residual is computed as:

$$r^{(m)} = f\big(s^{k,m}\big),$$

which corresponds to the call to the stencil operator `diffusion`. The convergence of the nonlinear iteration is monitored through the Euclidean norm of the residual:

$$\|r^{(m)}\|_2.$$

If the residual falls below a prescribed tolerance, the Newton iteration is terminated.
Otherwise, the Newton correction $\delta s^{(m)}$ is obtained by approximately solving the linearized system

$$J\big(s^{k,m}\big)\,\delta s^{(m)} = r^{(m)},$$

where J denotes the Jacobian of the residual. This system is solved using a matrix-free Conjugate Gradient method, implemented in `hpc_cg`, which avoids the explicit construction of the Jacobian matrix.

If the CG solver converges, the solution is updated according to the Newton step:

$$s^{k,m+1} = s^{k,m} - \delta s^{(m)},$$

implemented via the BLAS-like operation `hpc_axpy`.

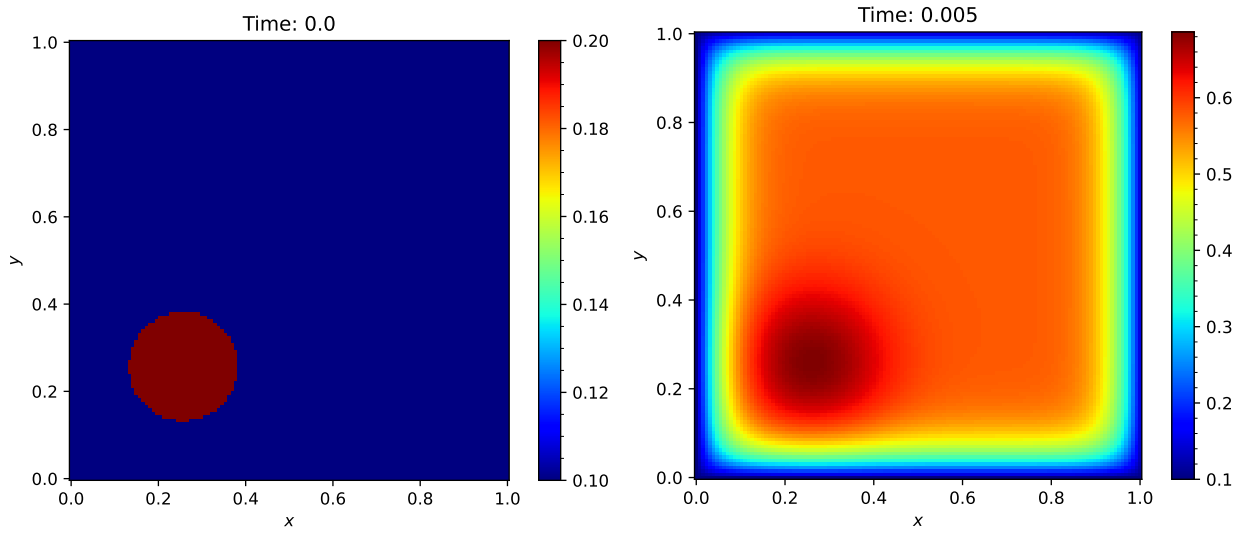
The process is repeated until either convergence is reached or a maximum number of Newton iterations is exceeded. If convergence is not achieved, the simulation is terminated and an error message is reported.

1.2. Results

The implementation of the linear algebra functions and the stencil operators was successful, as the program runs without errors and produces results consistent with the expected behavior of the solution to Fisher's equation.
In particular the output is given by the following listing

```
================================================================================
                    Welcome to mini-stencil!
version :: C++ Serial
mesh :: 128 * 128 dx = 0.00787402
time :: 100 time steps from 0 .. 0.005
iteration :: CG 300, Newton 50, tolerance 1e-06
================================================================================
simulation took 0.157575 seconds
1513 conjugate gradient iterations, at rate of 9601.76 iters/second
300 Newton iterations
--------------------------------------------------------------------------------
### 1, 128, 100, 1513, 300, 0.157575
```

Looking at the project handout and comparing with our results, we can see that the number of CG iterations and Newton iterations are consistent with the expected values, which indicates that our implementation is correct.
We can also see in figures 1a and 1b the initial and final solution of the problem, which are consistent with the images provided in the project handout, confirming the correctness of our implementation.



(a) Initial solution

(b) Final solution

Figure 1: Comparison between initial and final solution

2. Task: Adding OpenMP to the nonlinear PDE mini-app [50 Points]

2.1. Welcome message in `main.cpp` and serial compilation [5 points]

To solve this task we have modified the `main.cpp` file with the modifications shown in listing 1.

Listing 1: Modifications to the welcome message in `main.cpp` to support OpenMP

```
[...]

#ifdef _OPENMP
#include <omp.h>
#endif

[...]

#ifdef _OPENMP
int threads = omp_get_max_threads();
#endif
#ifndef _OPENMP
int threads = 1;
#endif

[...]

#ifdef _OPENMP
std::cout << "version :: C++ OpenMP" << std::endl;
#endif
#ifndef _OPENMP
std::cout << "version :: C++ serial" << std::endl;
#endif

[...]
```

2.2. Linear algebra kernel [10 Points]

All the functions in the file `linalg.cpp` were parallelized using the `#pragma omp parallel for` directive to parallelize the loops.
For the function `hpc_norm2` we have to take into account the reduction of the sum, which is done using the `reduction(+:sum)` clause.

2.3. The diffusion stencil [15 Points]

The parallelization of the `diffusion()` routine is based on OpenMP loop-level parallelism.
The largest amount of work is associated with the interior points, since they constitute almost the whole domain. For this reason, the interior stencil evaluation is parallelized with

```
#pragma omp parallel
{
    #pragma omp for collapse(2) nowait
    for (int j = 1; j < jend; j++) {
        for (int i = 1; i < iend; i++) {
            ...
        }
    }
    ...
}
```

The `#pragma omp parallel` directive creates a team of threads only once, and all subsequent loop worksharing constructs are executed inside the same parallel region. This is more efficient than creating a new parallel region for every loop separately.

For the interior update, the directive `#pragma omp for collapse(2)` is used. The `collapse(2)` clause merges the two nested loops over i and j into a single iteration space.

After the interior points, the four non-corner boundaries are treated in separate loops:

- east boundary,
- west boundary,
- north boundary,
- south boundary.

These loops are also parallelized with `#pragma omp for`. They are kept separate because the stencil on each boundary differs from the interior stencil due to the Dirichlet boundary values `bndE`, `bndW`, `bndN`, and `bndS`.

For the first boundary loops, the `nowait` clause is added:

```
#pragma omp for nowait
```

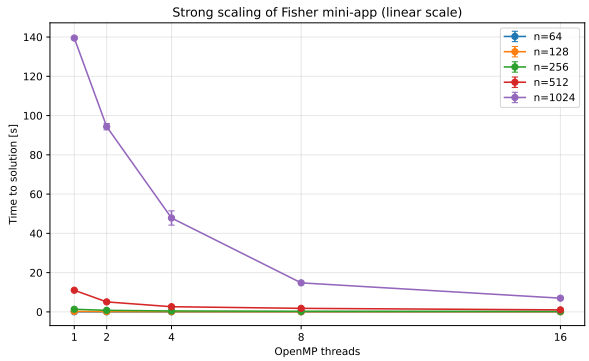
This removes the implicit barrier at the end of each loop. Since the different boundary updates are independent, there is no need for all threads to synchronize after every single boundary loop. Avoiding these unnecessary barriers reduces synchronization overhead and improves performance.

The four corner points are computed outside the parallel region and therefore serially. This choice is reasonable because only four grid points are involved, so the computational cost is negligible compared to the interior and edge updates. Parallelizing these few operations would introduce extra overhead without any meaningful speedup.

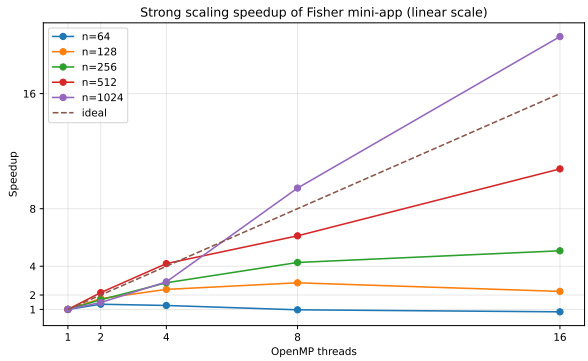
An important property of this implementation is that it is free of data races. Each iteration writes to a distinct entry of the residual field $f(i, j)$, while the fields `s_old`, `s_new`, and the boundary arrays are only read. Therefore, no two threads update the same memory location simultaneously.

2.4. Strong scaling [10 points]

In the strong scaling test we have measured the time to solution for different combinations of number of threads and problem sizes.
In particular, we have measured the time to solution for NCPU = 1, 2, 4, 8, 16 threads across resolutions of $n \times n$, where n = 64, 128, 256, 512, 1024.
The time to solution plot and the speedup plot are shown respectively in figures 2a and 2b.



(a) Time-to-solution.



(b) Speedup.

Figure 2: Strong scaling results on a linear scale.

The strong scaling results in figures **??** and **??** show a marked dependence on the problem size. For the smallest meshes, $n = 64$ and, to a lesser extent, $n = 128$, the scaling is poor and even deteriorates as the number of threads increases. In these cases, the time-to-solution is already very small for low thread counts, so the parallel overhead associated with OpenMP, such as thread management, synchronization, and barriers, becomes comparable to or larger than the useful computational work. As a consequence, the measured speedup remains limited and for $n = 64$ even drops below one, meaning that the parallel execution is slower than the serial one.

For the intermediate problem sizes, $n = 256$ and $n = 512$, the behavior improves significantly. The time-to-solution decreases as the number of threads increases, and the corresponding speedup is clearly positive.

However, the scaling is still sublinear and tends to flatten at larger thread counts. This is a typical strong scaling effect: since the total problem size is fixed, increasing the number of threads reduces the amount of work assigned to each thread, while synchronization costs and memory-access overhead become more relevant.

The largest problem, $n = 1024$, exhibits the best scaling performance. In this case, the computational workload is sufficiently large to amortize the parallel overhead, and the speedup continues to increase up to 16 threads. Moreover, the measured speedup slightly exceeds the ideal trend at the highest thread count. This superlinear behavior can be explained by improved cache efficiency: when the domain is divided among multiple threads, the working set handled by each thread becomes smaller and may fit better into cache, thereby reducing memory traffic compared with the single-thread execution.

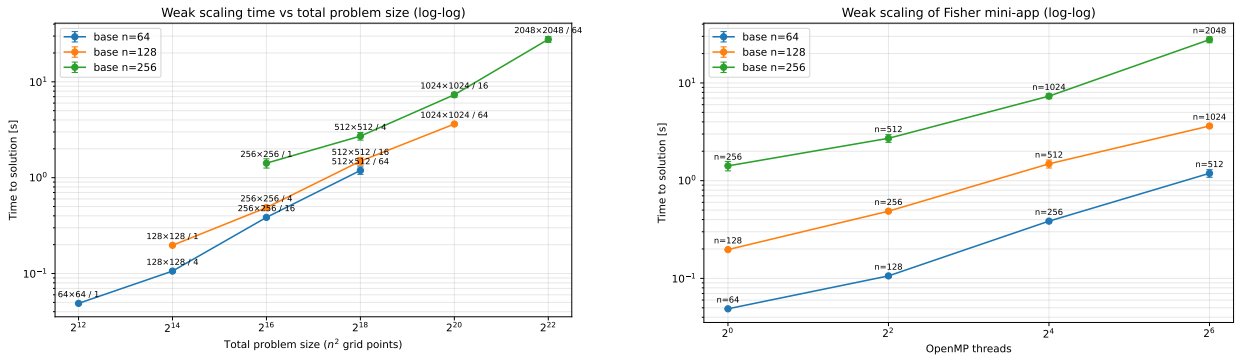
Overall, these results confirm that the effectiveness of strong scaling depends strongly on the granularity of the problem. Small grids do not provide enough work to exploit many threads efficiently, whereas larger grids benefit much more from parallelization. The departure from ideal scaling at higher thread counts is consistent with the combined effects of OpenMP overhead, memory bandwidth limitations, and the serial fraction of the algorithm.

To perfrom the strong scaling test every data reported in the plots of figures 2a and 2b is the average of 3 runs, and the error bars represent the standard deviation across these runs.

2.5. Weak scaling [10 points]

To assess the weak scaling behavior of the Fisher mini-app, we measured the time-to-solution while increasing the total problem size proportionally to the number of OpenMP threads, so as to keep the workload per thread approximately constant. In particular, the tests were performed with $N_{\mathrm{CPU}} = 1$, 4, 16, 64 and with three base resolutions $n = 64$, $n = 128$, and $n = 256$. For each base resolution, the grid size was increased according to the weak scaling requirement.

Figure 3 reports the weak scaling results in two complementary ways. Figure 3a shows the time-to-solution as a function of the total problem size, with annotations indicating the corresponding grid and thread count. Figure 3b shows the same runtimes as a function of the number of OpenMP threads for the three base resolutions.



(a) Time-to-solution.

(b) Speedup.

Figure 3: Weak scaling results on a linear scale.

The weak scaling results in Figure 3 show that the runtime does *not* remain constant as the number of threads and the problem size increase proportionally. For all three base resolutions, the time-to-solution increases with increasing thread count, which indicates a departure from ideal weak scaling. This means that, although the workload per thread is kept constant in terms of grid points, the total computational cost does not scale proportionally.

A first reason for this behavior is the nonlinear nature of the problem. As suggested in the exercise statement, the convergence of the Newton–CG solver depends on the spatial resolution. When the grid is refined, the number of nonlinear and linear iterations may increase, so the cost per thread is no longer strictly constant even if the number of grid points handled by each thread remains approximately the same. Therefore, part of the runtime growth is due to the numerical solver itself and not only to parallel overhead.

A second contribution comes from the parallel execution. As the number of threads increases, synchronization, reductions, and memory-access overhead become more relevant. These effects further prevent ideal

weak scaling, for which the runtime would be expected to stay nearly flat.

More specifically, the base case $n = 64$ exhibits the best absolute runtimes, but also shows a clear increase from the smallest to the largest configuration. The same qualitative trend is observed for base $n = 128$ and $n = 256$, with larger base resolutions leading to systematically larger runtimes. In particular, the $n = 256$ series is the most expensive and grows most markedly, indicating that the increased resolution significantly affects the total solution cost.

Overall, the results show that the code exhibits only limited weak scaling. The increase in runtime with thread count suggests that keeping the number of grid points per thread constant is not sufficient to maintain a constant time-to-solution for this application. The main reason is that the computational work is influenced not only by the local grid size per thread, but also by the resolution-dependent convergence of the nonlinear solver, together with the usual OpenMP overheads and memory effects.